

Hierarchical Admission Control for Opaque Multi-Tenant Managed LLM APIs

Anonymous Author(s)

Affiliation withheld for double-blind review

Abstract

Managed LLM APIs hide scheduler state, accelerator occupancy, batching, and instantaneous backend capacity, making both overload protection and multi-tenant isolation difficult at the application gateway. We present a hierarchical admission controller that first adapts a global in-flight budget using backend latency, demand, and token-weighted occupancy, then applies per-tenant and per-tenant-class admission budgets. On Amazon Bedrock with Llama 4 Maverick, demand-gated global control improves dynamic-load SLO-goodput by 15.5% over the best observed fixed cap, while token-aware control improves mean goodput by 152% under a constant-RPS token shift. Under noisy-neighbor load, tenant-aware admission raises protected-tenant goodput from 0.301 to 0.492 req/s and shifts overload shedding away from the protected tenant. Under mixed short/long traffic within one tenant, class-aware admission improves short-request goodput from 0.253 to 0.436 req/s. A uniform-overflow control confirms the class-isolation effect while showing that part of the apparent tenant-versus-class separation comes from queueing semantics. Under joint interference, the full hierarchy achieves the best protected-class goodput in 5/5 comparisons against tenant-only control and 4/5 against class-only control. These results show that opaque managed LLM gateways benefit from separating backend-capacity control from tenant- and workload-level allocation.

CCS Concepts

- Computer systems organization → Cloud computing;
- Software and its engineering → Software performance;
- Computing methodologies → Natural language generation

Keywords

managed LLM APIs, hierarchical admission control, SLO goodput, token-aware control, multi-tenant isolation

ACM Reference format:

Anonymous Author(s). 2027. Hierarchical Admission Control for Opaque Multi-Tenant Managed LLM APIs. In *Proceedings of the 42nd ACM Symposium on Applied Computing (SAC'27)*, April 5–9, 2027, Gwangju, South Korea. 8 pages.

1 Introduction

Managed LLM APIs are becoming a standard cloud inference substrate because they remove accelerator provisioning, model

deployment, and serving operations from application teams. Amazon Bedrock, for example, exposes regional and cross-Region inference while the service internally determines where requests execute [1]. The client observes request outcomes, token counts, latency, and throttling, but not provider queue depth, batch composition, GPU occupancy, or instantaneous model capacity. Bedrock also publishes quotas such as requests/tokens per minute (RPM/TPM) [2] and runtime metrics including TimeToFirstToken (TTFT) and invocation throttles [3], but these do not answer a gateway’s sub-minute control question: how many requests should be in flight now?

The distinction matters for interactive generation. Tail latency determines user-visible experience [6], while long request duration means even modest arrival rates can create overlapping work by Little’s law [7]. LLM requests are also heterogeneous: two calls at the same rate may carry radically different prompt and output-token demand. Server-side LLM systems address this with batching, KV-cache management, and disaggregation [9], [16]–[21], but a managed API hides those controls from its clients.

We study whether a thin cloud gateway can regulate an opaque managed LLM backend using only client-observable feedback, via two complementary control layers: a token-aware global controller adapts the backend-safe in-flight budget $C_g[k]$ over discrete control cycles k , while hierarchical admission limits how that budget is consumed across tenants and workload classes through per-tenant budget C_u and per-tenant-class budget $C_{u,c}$. Put simply, global control asks how much capacity is safe; hierarchical admission asks who may consume it. Our objective is SLO-goodput: successful requests whose user-facing TTFT, including gateway queueing, meets a latency target. The global controller reacts to backend TTFT rather than user TTFT, so queue wait — gateway-side demand, not necessarily backend saturation — never becomes a false signal to reduce $C_g[k]$.

A latency-only adaptive policy creates another failure mode: when offered demand is below the current limit, healthy latency can be misread as evidence that more concurrency is useful. We therefore gate additive increase on observed queue pressure, and add an in-flight token-pressure signal since request count alone ignores LLM request size — yielding a design that is demand-aware and token-aware without using quota utilization as a direct control input.

Our evaluation uses Amazon Bedrock Llama 4 Maverick through a US cross-Region inference profile [4]; the documented 800-RPM/600k-TPM values are treated only as static environmental context [2], not an estimate of usable concurrency. The paper makes four contributions: (1) an opaque-backend formulation separating quota, arrival rate, token occupancy, and usable concurrency; (2) a global adaptive capacity controller combining demand-gated SLO-AIMD with token-weighted pressure; (3) a hierarchical admission design layering a global budget $C_g[k]$ with static per-tenant (C_u) and per-tenant-class ($C_{u,c}$) budgets; and (4) a controlled multi-tenant evaluation — noisy-neighbor tenant interference, intra-tenant class interference, a causal overflow-semantics control, and joint tenant×class interference — showing compositional validation with modest but consistent gains.

The hierarchy is not proposed as a replacement for global adaptive control: global control estimates available capacity, while the hierarchy supplies the missing allocation mechanism once that capacity becomes scarce. E5 and E6 establish the need for the two allocation dimensions separately; E7 evaluates their composition rather than seeking a large additional gain.

2 Background and Motivation

Cross-Region inference routes Bedrock calls across destination Regions to absorb bursts [1], but this makes instantaneous placement and service contention opaque: published quotas constrain usage over time [2] and CloudWatch exposes aggregate metrics [3], but neither exposes the provider scheduler or a concurrency-capacity curve. Rate and concurrency also operate on different dimensions — RPM bounds arrivals over a window while concurrency bounds outstanding work, and average in-flight demand scales as $L=\lambda W$ by Little's law [7] — so a gateway can remain well below its RPM limit while still accumulating simultaneous slow calls; conversely, a larger C is harmless at light load because it is never exercised. This motivates runtime control rather than a fixed cap derived from quota.

Raw throughput can reward overload, so following SLO- and user-experience-oriented evaluation [8], [9], [14] we define SLO-goodput as the rate of successful requests whose user-facing TTFT — including gateway queueing — meets the target, with backend TTFT retained separately as a control signal:

$$G = \frac{|\{\text{successful requests with TTFT}_{\text{user}} \leq \text{SLO}\}|}{\text{experiment duration.}} \quad (1)$$

Adaptive concurrency is well established — AIMD gives simple additive-probing/multiplicative-backoff feedback [11], and Envoy's gradient controller reacts to latency samples [10] — but managed LLM APIs add two ambiguities: healthy latency does not imply a saturated limit, and prefill/decode work scales with input/output tokens rather than request count [19], [21], [25], since two calls at the same rate may carry radically different token demand that server-side batching, KV-cache, and

disaggregation systems [9], [16]–[21] address but a managed API hides from its clients. The controller must therefore distinguish healthy-but-unused from healthy-and-demanded capacity, using a request-cost signal beyond count alone.

3 Controller Design

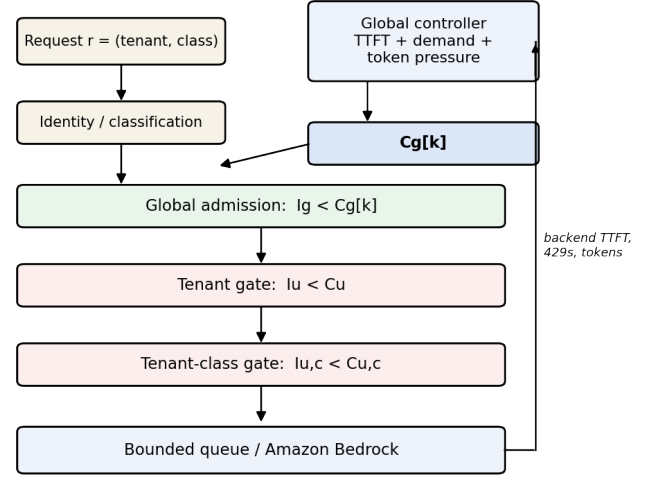


Figure 1: Two-layer control. Every 5 s the global controller uses backend TTFT, throttle/queue signals, and weighted in-flight tokens to set the safe budget $C_g[k]$ (§3.1–3.2); each request is classified by tenant/class and passed through the global, tenant, and tenant-class admission gates (§3.3) before a bounded queue to Amazon Bedrock. Static RPM/TPM are logged for context, not control inputs.

3.1 Demand-gated SLO-AIMD

The base policy applies additive increase when the backend is healthy and multiplicative decrease under pressure ($\beta=0.7$), where pressure is backend P95 TTFT above the SLO or provider throttling/error feedback. Healthy latency alone is not sufficient for an increase: additive increase requires the current limit to be demand-saturated — waiting requests, a positive max-wait count, or queue P95 above 5 ms — otherwise the policy holds C . This demand gate prevents vacant-limit drift during light load.

$$C_g[k+1] = C_g[k] + 1 \text{ if } \text{healthy} \wedge \text{demand}; \max(1, 0.7 \cdot C_g[k]) \text{ if pressure; } C_g[k] \text{ otherwise.} \quad (2)$$

3.2 Token-aware pressure

Request-count concurrency treats all calls as interchangeable. For request i we instead assign a token weight of input tokens plus $\lambda \times$ estimated output tokens ($\lambda=1$), summed over in-flight calls, and compare the weighted occupancy W_k to the reference $C_g[k] \cdot w_{\text{short}}$ ($w_{\text{short}}=512+128=640$). If the ratio exceeds $\gamma=1.2$ the controller decreases C ; if above 1 while the base controller would increase, it holds instead. The output-token term uses the request's declared max_tokens — resolved from the short/long class preset (128/512) when unset — known before

invocation, never actual generated output. We use Bedrock token accounting [5] to build the weight; the signal is an occupancy proxy, not TPM/quota control.

$$W_k = \sum_{i \in \text{inflight}} (\text{tokens_in}, i + \lambda \cdot \text{tokens_out}, i^{\text{est}}), \lambda = 1. \quad (3)$$

3.3 Hierarchical admission

Global control decides how much in-flight work is safe; it does not decide who may use it. We add a lightweight admission layer on top of $C_g[k]$. Each request $r=(u,c)$ belongs to a tenant u and a workload class c (short or long). The gateway maintains three in-flight counters, I_g , I_u , and $I_{\{u,c\}}$, and three budgets: the dynamic global budget $C_g[k]$ from Sections 3.1–3.2, a static per-tenant budget C_u , and a static per-tenant-class budget $C_{\{u,c\}}$.

$$\text{Admit } r=(u,c) \text{ iff } I_g < C_g[k] \wedge I_u < C_u \wedge I_{\{u,c\}} < C_{\{u,c\}}. \quad (4)$$

All three counters increment on admission and decrement together when the request completes. $C_g[k]$ is dynamic and comes from the token-aware global controller; C_u and $C_{\{u,c\}}$ are static isolation budgets. The hierarchy does not estimate GPU capacity or provider scheduler state; it only partitions gateway-visible in-flight capacity already judged safe. Under queued operation, tenant and tenant-class violations are rejected immediately (`tenant_full` / `tenant_class_full`), whereas a request blocked only by the global limit may wait and time out (`queue_timeout`); Section 5.4 shows why this matters, and P0 later equalizes these overflow semantics.

4 Experimental Methodology

The evaluation answers five questions. RQ1 asks whether one fixed C describes usable capacity across loads (E1/E2). RQ2 asks whether adaptive global control can exploit transient capacity during a burst (E3). RQ3 asks whether token awareness helps when RPS is unchanged but request cost shifts (E4). RQ4 asks whether tenant-aware admission protects a tenant from a noisy neighbor (E5, and a causal overflow-reject control P0). RQ5 asks whether tenant isolation alone is sufficient for mixed workload classes, and whether the tenant and class dimensions compose (E6, P0, and E7). This ordering separates characterization, dynamic control, LLM-specific heterogeneity, and multi-tenant allocation.

4.1 Platform and workload

We evaluate Amazon Bedrock (us-east-1, US cross-Region profile for Llama 4 Maverick 17B Instruct [1], [4]); streaming invocation makes TTFT directly observable [3]. The short/reference profile is ~512 input/128 output tokens, the long phase ~4096/512, with deterministic generation. Unless noted otherwise, experiments use a 5-s control window, $\beta=0.7$, a 2-s queue timeout, a 576-ms TTFT SLO, and $R_{\text{ref}}=1.84$ req/s; the

documented 800-RPM/600k-TPM limits are environmental context only [2].

4.2 Characterization, baselines, and open-loop runs

E1 is a closed-loop scout at $C \in \{1, 2, 4, 8\}$ that derives experiment knobs, not a universal Bedrock capacity: the best observed point is $C^*=1$, giving $R_{\text{ref}} \approx 1.84$ req/s, P95 TTFT=384 ms, and a $1.5 \times$ TTFT SLO of 576 ms — a fixed evaluation threshold to distinguish healthy from degraded operation, not a claim that 576 ms is a universal user-facing SLO. E2 checks $0.5R_{\text{ref}}$ light-load behavior (5 reps). Main dynamic experiments use open-loop arrivals — so offered load does not decrease when responses slow — and compare Fixed-1, an Envoy-style gradient baseline [10], and demand-gated SLO-AIMD; token-shift experiments compare request-count SLO-AIMD and Token SLO-AIMD. E3 runs 420 s ($0.5/0.9/1.5/0.6 \times R_{\text{ref}}$ phases); E4 holds $0.9R_{\text{ref}}$ for 480 s while request size shifts short→long→short. Both use 5 reps; phase statistics are computed from per-request arrival times, with 1 s series smoothed over 5–10 s only for visualization.

4.3 Metrics and ablation

We report SLO-goodput, TTFT percentiles, provider throttles, mean C , phase goodput, and recovery time, using means and medians since five runs do not support strong distributional claims. A three-repetition signal ablation on the E4 workload isolates token, TTFT, throttle, and multiplicative-decrease feedback (end of §5.3); a separate quota-pressure run is excluded because the gateway itself, not the provider quota, was the bottleneck.

4.4 Multi-tenant admission workloads

E5–E7 and the P0 control all run on top of the E4 token-aware global controller $C_g[k]$; only the admission budgets vary. E5 (tenant isolation, 5 reps, 420 s) uses two tenants, A and B, both issuing only short requests: A holds at $0.5R_{\text{ref}}$ throughout, while B bursts to $0.9R_{\text{ref}}$ during P2 (120–300 s). E6 (class isolation, 5 reps, 420 s) uses a single tenant whose P2 traffic mixes 70% short and 30% long requests at $0.9R_{\text{ref}}$. E7 (joint interference, 5 reps, 420 s) combines both: in P2, tenant A shifts to $0.5R_{\text{ref}}$ with an 80%/20% short/long mix while tenant B bursts to $0.7R_{\text{ref}}$ with a 50%/50% mix; P1 and P3 hold A at $0.5R_{\text{ref}}$, 100% short, with B silent, so protected-class goodput depends on both isolation layers together. E5 and E6 compare Global, Tenant-only, Class-only, and Hierarchical admission under bounded queueing; E7 compares Tenant-only, Class-only, and Hierarchical admission under joint interference. Tenant and tenant-class budgets are fixed before each experiment, not tuned per repetition: the global ceiling is $C_g^{\text{max}}=2$ with tenant caps $C_A=2$, $C_B=1$; Hierarchical further splits these into per-tenant-class caps $C_{\{A,\text{short}\}}=2$, $C_{\{A,\text{long}\}}=1$, $C_{\{B,\text{short}\}}=1$, $C_{\{B,\text{long}\}}=1$, while Class-only uses a shared global class pool

$C_{\text{short}}=2$, $C_{\text{long}}=1$ that cannot distinguish tenants. P0 (overflow-reject control, 3 reps, 420 s) reruns the E5/E6 policies with `overflow_mode=reject` and `queue_max=0`, so every rejection is an immediate `*_full` event rather than a `queue_timeout`; this isolates the isolation effect from queueing-artifact effects. Primary metrics are P2 per-tenant-class goodput, not whole-run aggregates, since P1 and P3 goodput are similar (~0.9) across policies in every experiment.

5 Evaluation

5.1 RQ1: Load-dependent usable concurrency

Table 2 shows two complementary regimes. Under the E1 workload, increasing C from 1 to 2 lowers SLO-goodput from 1.84 to 1.04 req/s and raises P95 TTFT from 384 to 3181 ms, with more throttles and no throughput recovery, so $C^*=1$ is the best observed point here, not a universal Bedrock limit. At $0.5R_{\text{ref}}$ in E2, however, Fixed-2 is as healthy as Fixed-1 (0.913 vs. 0.911 goodput, 377 vs. 434 ms P95, zero throttles): usable concurrency depends on offered demand.

Table 2: Characterization and light-load sanity. E1 is one scout repetition; E2 reports five-run means.

Run	Policy/C	Goodput	P95 TTFT	429s/run
E1	C=1	1.84	384 ms	0
E1	C=2	1.04	3181 ms	1
E1	C=4	0.90	9381 ms	6
E1	C=8	0.81	8282 ms	31
E2	Fixed-1	0.911	434 ms	0
E2	Fixed-2	0.913	377 ms	0
E2	SLO-AIMD	0.892	468 ms	0

E2 also validates the demand gate: revised SLO-AIMD holds mean C at exactly 1.00, matches the best fixed policies within normal run-to-run spread, and generates no provider throttles (an earlier diagnostic controller without the gate let vacant C climb despite only 0–1 in-flight requests). A three-repetition E1 replication preserves the $C=1$ ordering, though P95 TTFT varies across runs, so we retain the pre-specified R_{ref} /SLO calibration rather than redefining it post hoc.

5.2 RQ2: Dynamic-load adaptation

Table 3 summarizes five-run E3 means. Fixed-1 achieves 1.038 req/s whole-run SLO-goodput; the generic gradient controller improves this 13% to 1.173 req/s; demand-gated SLO-AIMD reaches 1.199 req/s, a 15.5% gain, while keeping mean C at 1.38 and averaging 0.4 throttles/run. Whole-run P95 TTFT is effectively a tie (2140 vs. 2173 ms), so the result is a goodput gain, not a tail-latency claim.

Table 3: E3 dynamic-load whole-run means over five repetitions.

Policy	Goodput vs Fixed	P95 ms	429s/rep	Mean C
Fixed-1	1.038 —	2173	0.0	1.00
Gradient	1.173 +13%	2306	1.2	2.77
SLO-AIMD	1.199 +15.5%	2140	0.4	1.38

The mechanism is clear by phase (Figure 2, left): P1/P2 behavior is nearly identical across policies, but during the $1.5R_{\text{ref}}$ burst (P3) Fixed-1 collapses to 0.127 req/s while Gradient and SLO-AIMD reach 1.117 and 1.243 — about $9.8\times$ Fixed-1's burst goodput — and P4 converges again. The gain comes from temporarily exploring $C=2-3$ only when demand is queued and TTFT stays healthy, not from permanently raising the limit.

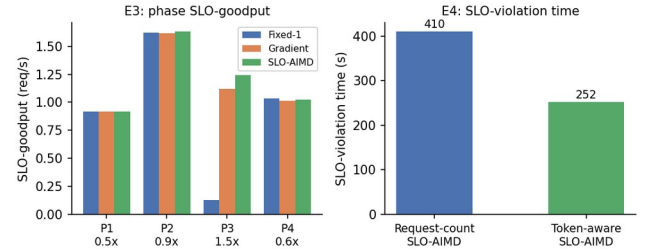


Figure 2: E3 phase SLO-goodput (left) and E4 SLO-violation time under token-aware vs. request-count control (right).

5.3 RQ3: Constant-RPS token shift

E4 holds RPS at $0.9R_{\text{ref}}$ while request size changes. Request-count SLO-AIMD achieves mean goodput 0.321 req/s (median 0.228, mean $C=2.07$); Token SLO-AIMD reaches 0.809 req/s (median 0.831) — 152% mean, 264% median improvement. One request-count repetition is a high outlier, so the median is the more representative aggregate.

The important result is recovery, not long-phase goodput. During P2 the token-aware controller intentionally pins $C=1$ and rejects ≈ 148 long requests/run, keeping P2 goodput near zero but preventing 4608-token requests from accumulating occupancy; in P3, Token SLO-AIMD reaches 1.60 req/s, 98% SLO attainment, 432 ms P95. Request-count SLO-AIMD instead grows C toward 6–7 during long occupancy and returns only 0.14 req/s in P3.

Table 4: E4 constant-RPS token shift. Two request-count runs never recovered.

Metric	SLO-AIMD	Token-aware
Whole-run mean	0.321	0.809
Whole-run median	0.228	0.831
P3 short goodput	0.14	1.60
Recovery success	3/5	5/5
Recovery time	6.8/11.8/76.5 s	16.8 s mean
Violation time	410 s	252 s

Recovery is the first three consecutive 1 s windows with user TTFT $P95 \leq SLO$ after $t=300$ s. Token-aware control recovers in all five runs (mean 16.8 s); request-count SLO-AIMD recovers in only three (6.8, 11.8, 76.5 s; two never recover). Total SLO-violation time falls from 410 s to 252 s (Figure 2, right). Offered token rate stays below the static 600k-TPM context, so the mechanism is token occupancy, not a quota-percentage trigger.

A signal ablation further supports the token mechanism: on the same workload, removing token awareness reduces mean goodput by 73% (0.256 vs. 0.946). Removing TTFT feedback mainly increases provider throttling (12.7 Bedrock 429s/rep) rather than changing the headline goodput; throttle feedback and multiplicative decrease are not identifiable in this low-C operating range.

5.4 RQ4: Tenant isolation

Under the queued E5 workload, tenant-aware admission raises protected tenant A’s short-request SLO-goodput from 0.301 req/s under Global to 0.492 under Tenant-only, and 0.509 under Hierarchical. The more informative effect is where overload is shed: Global rejects 62 A and 81 B requests during the P2 burst, whereas Tenant-only rejects only 9 A but 135 B. Tenant-aware admission thus redirects overload away from the protected tenant rather than merely raising aggregate throughput, without starving B (queued P2 goodput ≈ 0.68).

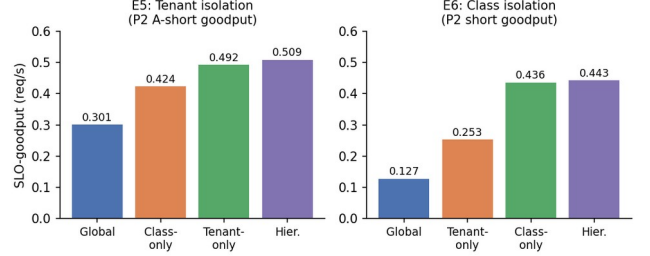


Figure 3: P2 protected-class SLO-goodput under E5 tenant isolation (left) and E6 class isolation (right).

However, the uniform immediate-rejection control (P0) narrows the Tenant-only versus Class-only gap to 0.453 vs. 0.442 req/s, with Tenant-only higher in only 2 of 3 reps — so we do not claim categorical superiority of tenant-aware over class-aware admission in E5. The more robust E5 contrast is Tenant-only versus Global (≈ 0.45 versus 0.34 req/s); part of the large queued tenant-vs-class separation is attributable to overflow semantics rather than tenant isolation itself.

5.5 RQ5: Class isolation and hierarchy

With one tenant carrying both short and long requests, tenant identity cannot separate the two workloads. Under queued semantics (E6), short-request goodput rises from 0.253 req/s (Tenant-only) to 0.436 (Class-only) and 0.443 (Hierarchical) — Figure 3, right. Global performs worst of all (0.127 req/s): same-tenant long traffic wrecks short-request goodput when no layer distinguishes classes.

The uniform-overflow control (P0) preserves the class effect: Class-only exceeds Tenant-only in all three reps (0.545 vs. 0.518 req/s), while Tenant-only is close to Global (0.518 vs. 0.504). This shows the large queued Global-to-Tenant gain in E6 is partly an overflow artifact, whereas the incremental class-isolation effect survives after overflow semantics are equalized — the identified gap is class vs. tenant, not tenant vs. global.

Policy	E5 GA-short	E6 Gshort
Global	0.338	0.504
Class-only	0.442	0.545
Tenant-only	0.453	0.518
Hierarchical	0.427	0.450

Table 5: P0 overflow-reject control, P2 protected-class SLO-goodput (mean of 3 reps).

P0 is a causal control, not an end-to-end ranking. Immediate rejection makes compound caps more conservative — any one of

the global, tenant, or tenant-class gates can turn a request away with no chance to wait — so Hierarchical, which stacks the most gates, need not win here (E5: 0.427 vs. Tenant-only 0.453; E6: 0.450, below Global 0.504). We use P0 to identify the Tenant-only vs. Class-only contrasts, not to claim Hierarchical must dominate under reject-only overflow.

Under joint tenant and class interference (E7), Tenant-only, Class-only, and Hierarchical admission achieve protected A-short goodput of 0.183, 0.213, and 0.234 req/s: Hierarchical wins all five paired reps against Tenant-only and four of five against Class-only. Because the gaps are modest, we treat E7 as compositional validation that the two layers combine sensibly, not as primary evidence for either — that comes from E5 and E6.

6 Discussion and Limitations

The results expose two distinct gateway responsibilities that neither layer subsumes: global token-aware control estimates how much provider concurrency is currently safe, while tenant and tenant-class admission determine how that scarce capacity is allocated. E5 shows cross-tenant interference; E6 shows that tenant isolation alone cannot distinguish short and long workloads within the same tenant. This mirrors E4's deliberate short-vs-long trade-off (§5.3), which is appropriate for an SLO-goodput objective but not for every service class.

Our objective is protected-class SLO-goodput, not equal request-level fairness: hierarchical admission may intentionally shed more of a noisy neighbor's requests to preserve interactive traffic. In E5, Global rejects 62 A and 81 B requests while Tenant-only rejects 9 A but 135 B — a deliberate priority choice, not an accident, and B is not starved (queued P2 goodput ≈ 0.68).

6.1 Overflow Semantics Matter

E5 and E6 also show that overload handling is itself a control variable. Immediate admission rejection can outperform bounded queueing under severe overload even when the policy lacks the relevant tenant or class information — this is why the queued Tenant-only versus Global gap partly collapses under P0. Our overflow-reject control therefore equalizes overflow semantics to distinguish isolation effects from queueing effects, which we recommend as standard practice when evaluating gateway admission policies.

6.2 Limitations

The study has four limitations. First, it evaluates one provider and one model family, so absolute C, R_ref, and budget values must not be generalized. Second, the E1 calibration comes from a one-repetition scout; a three-repetition replication confirms the $C=1 > C=2 > C=4$ ordering in SLO-goodput but P95 TTFT varies substantially across runs, so we retain the pre-specified calibration rather than redefining it post hoc. Third, tenant and tenant-class budgets are static and not yet dynamically learned. Fourth, we evaluate only short and long service classes with one linear token-weight model that uses the declared max_tokens

rather than actual generated length — a conservative design choice favoring a deployment-available estimate over utilization, with the hidden Bedrock backend also preventing attribution to specific GPU or scheduler causes. Dynamic tenant-budget adaptation, richer service classes, cross-provider replication, and learned output-cost prediction are natural next steps.

7 Related Work

General and LLM-specific serving systems establish feedback-driven, goodput-aware serving under full infrastructure control, spanning latency-aware prediction serving [12], [13], goodput optimization under unpredictable load [14], [15], and LLM-specific scheduling, memory management, prefill/decode handling, disaggregation, and QoE refinement [9], [16]–[25]. All of these systems assume direct visibility into, or control over, the serving infrastructure; our work is orthogonal, placing the controller at the application-to-managed-service boundary instead.

Cocktail [15] and SHEPHERD [14] address multi-tenant allocation but still assume infrastructure control. Closer to our setting, VTC [26] performs token-cost-aware fair scheduling inside an LLM serving system, and FairServe [27] combines application-aware throttling with service-side scheduling on a real multi-tenant Copilot workload; both still assume in-system control such as scheduler priority or KV-cache partitioning. A managed API client has none of these levers: it can only decide which requests to admit past its own gateway. Our controller therefore sits one layer above existing multi-tenant serving work, treating isolation as an admission-control problem rather than a scheduling problem.

8 Conclusion

Managed LLM gateways face two coupled problems: estimating a safe global in-flight budget without backend visibility, and preventing tenants or workload classes from monopolizing it. Demand- and token-aware global control improves SLO-goodput under dynamic and token-shifting workloads; hierarchical tenant and tenant-class admission adds isolation under noisy-neighbor and mixed-class interference. Controlled overflow experiments show queueing semantics can confound apparent isolation gains, motivating causal controls in gateway admission evaluation. The results support a layered design in which adaptive global control determines how much capacity is safe and hierarchical admission determines who may consume it.

References

- [1] Amazon Web Services. 2026. Route model inference requests across AWS Regions with cross-Region inference. Amazon Bedrock User Guide.
- [2] Amazon Web Services. 2026. Amazon Bedrock endpoints and quotas. AWS General Reference.

- [3] Amazon Web Services. 2026. Monitor bedrock-runtime inference using CloudWatch metrics. Amazon Bedrock User Guide.
- [4] Amazon Web Services. 2026. Llama 4 Maverick 17B Instruct. Amazon Bedrock User Guide.
- [5] Amazon Web Services. 2026. How tokens are counted in Amazon Bedrock. Amazon Bedrock User Guide.
- [6] Jeffrey Dean and Luiz André Barroso. 2013. The Tail at Scale. *Communications of the ACM* 56, 2 (2013), 74–80.
- [7] John D. C. Little. 1961. A Proof for the Queuing Formula: $L = \lambda W$. *Operations Research* 9, 3 (1961), 383–387.
- [8] Amey Agrawal et al. 2024. Etalon: Holistic Performance Evaluation Framework for LLM Inference Systems. *arXiv:2407.07000*.
- [9] Yinmin Zhong et al. 2024. DistServe: Disaggregating Prefill and Decoding for Goodput-Optimized Large Language Model Serving. In *OSDI 2024*, 193–210.
- [10] Envoy Project. 2026. Adaptive Concurrency Filter. Envoy Proxy Documentation.
- [11] Dah-Ming Chiu and Raj Jain. 1989. Analysis of the Increase and Decrease Algorithms for Congestion Avoidance in Computer Networks. *Computer Networks and ISDN Systems* 17, 1 (1989), 1–14.
- [12] Daniel Crankshaw et al. 2017. Clipper: A Low-Latency Online Prediction Serving System. In *NSDI 2017*, 613–627.
- [13] Daniel Crankshaw et al. 2020. InferLine: Latency-Aware Provisioning and Scaling for Prediction Serving Pipelines. In *SoCC 2020*, 477–491.
- [14] Hong Zhang, Yupeng Tang, Anurag Khandelwal, and Ion Stoica. 2023. SHEPHERD: Serving DNNs in the Wild. In *NSDI 2023*, 787–808.
- [15] Jashwant Raj Gunasekaran et al. 2022. Cocktail: A Multidimensional Optimization for Model Serving in Cloud. In *NSDI 2022*, 1041–1057.
- [16] Gyeong-In Yu, Joo Seong Jeong, Geon-Woo Kim, Soojeong Kim, and Byung-Gon Chun. 2022. Orca: A Distributed Serving System for Transformer-Based Generative Models. In *OSDI 2022*.
- [17] Woosuk Kwon et al. 2023. Efficient Memory Management for Large Language Model Serving with PagedAttention. In *SOSP 2023*, 611–626.
- [18] Zhuohan Li et al. 2023. AlpaServe: Statistical Multiplexing with Model Parallelism for Deep Learning Serving. In *OSDI 2023*, 663–679.
- [19] Amey Agrawal et al. 2024. Taming Throughput-Latency Tradeoff in LLM Inference with Sarathi-Serve. In *OSDI 2024*, 117–134.
- [20] Biao Sun et al. 2024. Llumnix: Dynamic Scheduling for Large Language Model Serving. In *OSDI 2024*, 173–191.
- [21] Pratyush Patel et al. 2024. Splitwise: Efficient Generative LLM Inference Using Phase Splitting. In *ISCA 2024*, 118–132.
- [22] Jiachen Liu, Jae-Won Chung, Zhiyu Wu, Fan Lai, Myungjin Lee, and Mosharaf Chowdhury. 2024. Andes: Defining and Enhancing Quality-of-Experience in LLM-Based Text Streaming Services. *arXiv:2404.16283*.
- [23] Lianmin Zheng et al. 2024. SGLang: Efficient Execution of Structured Language Model Programs. In *NeurIPS 2024*.
- [24] Bingyang Wu et al. 2023. Fast Distributed Inference Serving for Large Language Models. *arXiv:2305.05920*.
- [25] Vikranth Srivatsa, Zijian He, Reyna Abhyankar, Dongming Li, and Yiying Zhang. 2025. Preble: Efficient Distributed Prompt Scheduling for LLM Serving. In *Proc. ICLR 2025*.
- [26] Ying Sheng, Shiyi Cao, Dacheng Li, Banghua Zhu, Zhuohan Li, Danyang Zhuo, Joseph E. Gonzalez, and Ion Stoica. 2024. Fairness in Serving Large Language Models. In *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24)*, 965–988.
- [27] Redwan Ibne Seraj Khan et al. 2024. Ensuring Fair LLM Serving Amid Diverse Applications. *arXiv:2411.15997*.